Speculative Decoding Methods

How different inference methods generate and verify tokens

1 Autoregressive Decoding

Single-step generation (no speculation)

1 Input Prompt / Context

Where | is | the | capital | of | China?

Target LLM (Large Model) — KV Cache

2 Generate next token (one at a time)

t: Where | is | the | capital | of | China? | The

t+1: Where | is | the | capital | of | China? | The | capital

t+2: Where | is | the | capital | of | China? | The | capital | of

t+3: Where | is | the | capital | of | China? | The | capital | of | China

3 Output (all tokens are accepted)

... | of | China? | The | capital | of | ...

Pros: Simple, stable, no extra components

Cons: Slow (1 token per forward pass)

Best for: General use, low repetition

2 Draft-Model Speculative Decoding

Use a small draft model to propose multiple tokens

1 Input Prompt / Context

Where | is | the | capital | of | China?

Draft Model (Small) — Propose k tokens

The | capital | of | Japan | is

Candidate Proposal (k=5)

Target LLM (Large Model) — KV Cache

2 Verification by target model

The | capital | of | Japan | is

Accepted | Rejected

Resume from last accepted token

Target LLM (Large Model) — KV Cache

3 Continue generation

... | of | China? | The | capital | of | ...

Pros: Higher throughput than autoregressive

Cons: Needs auxiliary model, verification overhead, rejections waste compute

Best for: General workloads with moderate repetition

3 Prompt Lookup Decoding

Retrieve next tokens from repeated n-grams in prompt

1 Input Prompt / Context

Where | is | the | capital | of | China?

2 Candidate Proposal via n-gram Lookup

n-gram index (built from prompt)

Matches for prefix "... capital of China?"

- ... China? The capital of China is Beijing.
- ... China? The capital of China remains Beijing.
- ... China? The capital of China, Beijing, is ...

Candidate next tokens (from matches)

The | capital | of | Japan | is

3 Verification by target model

Target LLM (Large Model) — KV Cache

The | capital | of | Japan | is

Accepted | Rejected

4 Continue generation from last accepted token

... | of | China? | The | capital | of | ...

Pros: No auxiliary model, leverages prompt repetition

Cons: Lookup may be sparse, still needs verification

Best for: Prompts with repeated n-gram patterns

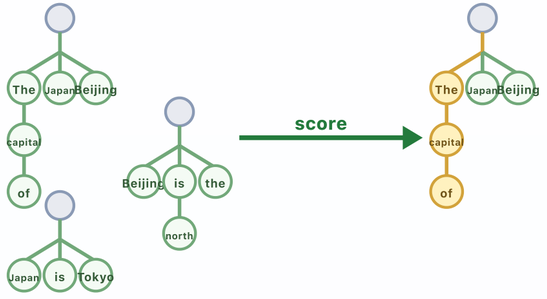
4 Suffix Speculative Decoding (Our Focus)

Reuse repeated substrings via suffix matches (no extra model)

1 Input Prompt / Context

Where | is | the | capital | of | China?

2 Candidate Proposal via Suffix Tree



3 Verification by target model

Target LLM (Large Model) — KV Cache

The | capital | of | Japan | is

Accepted | Rejected

4 Continue generation from last accepted token

... | of | China? | The | capital | of | ...

Pros: No auxiliary model, reuses context repetition, cache-friendly. Strong on repetitive workloads

Cons: Verification overhead, effectiveness depends on repetitions

Best for: Repetitive / templated content, code, docs, logs, SQL

Legend

- Prompt / Context (Already given)
- Candidate Tokens (From proposal)
- Accepted Tokens (Kept by target model)
- Rejected Tokens (Discarded)
- Cache / Memory (KV cache, index, tree)
- Target LLM (Large Model)
- Draft Model (Small Model, method ⊙)
- Data Flow
- Lookup / Retrieval
- Resume / Continue